

Security Vulnerabilities and Countermeasures of Smart Contracts: A Survey

Epherem Merete Sifra
School of Cyber Engineering
Xidian University
Xi'an, China
epherem@stu.xidian.edu.cn

Abstract— Recently, blockchain-based smart contracts have become popular because of their immunity, decentralization, and security insurance in a trustless environment. However, they are highly susceptible to various security risks, so it is necessary to systematically investigate existing security countermeasures and use unified evaluation criteria. There have been some surveys on the security of blockchain-based smart contracts, albeit they have failed to conduct a reasonable analysis of existing security solutions with a unified criterion. Therefore, this article comprehensively investigates the latest work on security of blockchain-based smart contracts. For better understanding, we will design a security model that represents blockchain-based smart contracts. Furthermore, we analyze the security requirements of blockchain and smart contracts and used these requirements as evaluation criteria to study the works under investigation. Finally, based on the results of the analysis, we present a series of open research issues and future directions to stimulate research work on protecting blockchain-based smart contracts.

Keywords— Blockchain, Smart Contracts, Security, Privacy, Cryptocurrency, Vulnerability

I. INTRODUCTION

Nick Szabo proposed the concept of smart contracts in 1996 [1], he defined it as computer programs designed to facilitate, verify, and enforce legal contracts and agreements in the digital realm. A classical smart contract relied on a centralized trusted party; as a result, it had a single point of failure. Blockchain is a decentralized, append-only Distributed Ledger Technology (DLT) that stores data on decentralized nodes. When Satoshi Nakamoto [2] designed the first blockchain and deployed a decentralized cryptocurrency, academia and industry immediately became interested in its decentralization, immutability, integrity, and reliability properties. These blockchain properties make it a great platform for smart contract deployment. Blockchain-based smart contracts provide trustworthy execution without a trusted third party (TTP). It eliminates the security risks posed by unreliable or unsecure centralized parties. Despite the popularity of smart contracts based on blockchain technology, they are particularly susceptible to several cyber-attacks. For instance, an adversary can exploit smart contract flaws to illicitly get benefits. Several real-world attacks, such as the attacks against Ethereum Decentralized Autonomous Organization (DAO) in 2016 [3] and the Parity Wallet hacks in 2017, target blockchain-based smart contracts. The DAO attack resulted in the loss of \$60 million worth of ether at the exchange rate at the time. This attack was caused by a flaw in the recursive call bug in the smart contract. This attack also forced the Ethereum network to perform a hard fork. To mitigate the dangers, it is crucial to analyze the security vulnerabilities, privacy issues, and their solutions in blockchain-based smart contracts in a systematic manner.

Several studies have examined the security of smart contracts [4] [5] [6] [7] [8]. However, they have not conducted a comprehensive investigation into blockchain-based smart contracts and potential security vulnerabilities. This research examines blockchain-based smart contract attacks and countermeasures. We explore blockchain-based smart contract attacks, provide security requirements, and utilize these requirements to analyze smart contract security countermeasures. Finally, we discuss open issues and future research directions.

The rest of our paper are organized as follows, Section II highlights potential attacks and the proposed security approach. Section III analyzes security countermeasures. Section IV identifies a series of open research issues and future directions. Section V is the conclusion of the paper.

II. SECURITY MODEL

In this section, we design a security model by analyzing the architecture of the public blockchain system.

In our security model, there is no fully trusted central authority responsible for system administration. Due to the fact that all nodes on the blockchain are logical and benefit-driven, nodes engaging in block mining cannot guarantee their trustworthiness. Additionally, blockchain nodes do not trust one another, and all nodes make judgments based on their own benefits and the data stored on the blockchain. Moreover, an adversary might control a portion of blockchain nodes. As a result, the adversary can broadcast numerous illegal transactions or blocks around the network. Similarly, an adversary's smart contract may initiate attacks on other smart contracts by exploiting a smart contract vulnerability. Finally, we examine the adversary's capacity to initiate a sequence of attacks, as described below.

A. Potential Attacks

In our security model, we consider an adversary capable of carrying out a series of attacks. To better demonstrate, we categorize potential attacks into three groups, i.e., Attacks on the blockchain network, Attacks on the consensus mechanism, and Attacks on smart contracts. In TABLE I we summarized some potential attacks that targeted the above three categories.

B. Security Requirements

In this subsection, we will assess the potential security requirements that must be fulfilled in blockchain-based smart contract attack countermeasures.

1) *Integrity (Int)*: Once smart contract deployments, smart contract calls, and transactions are all included in a new generated block, there must be consistency and no chance of compromise.

2) *Availability (Av)*: In the event of a denial of service (DoS) or distributed denial of service (DDoS) attack on a blockchain network, nodes and new or old blocks in the blockchain must always be accessible.

3) *Non-repudiation(Nr)*: The transaction sent by the clients and the block generated by each node are undeniable.

4) *Confidentiality (Cf)*: It is imperative that the details of transactions and the confidential information associated with smart contracts not be shared with unauthorized third parties.

5) *Correctness (Co)*: The smart contract code should always be correct; otherwise, slight errors in the code might lead to significant losses.

6) *Robustness*: The security solution should be able to withstand a variety of security attacks against smart contracts implemented on the blockchain.

TABLE I. THE CLASSIFICATIONS OF POTENTIAL ATTACK AND EXISTING SOLUTION SUMMARY

Attacks target	Potential attacks
Attacks on Network	DDoS Attack
	Eclipse Attack
	Sybil Attack
Attacks on Consensus	Selfish Mining
	Double-spend Attack
	51%(Majority) Attack
Attacks on Smart Contracts	Reentrancy
	Call Depth Attack
	DoS (Denial of Service)
	Timestamp Dependence
	Transaction Ordering Dependence (TOD)
	Mishandled Exception

III. COUNTERMEASURES

This section analyses existing security solutions using the provided security requirement as a criterion. The countermeasures are categorized into three groups according to their design goals: network-based, consensus-based, and smart contracts-based.

A. Network Attacks Solutions

Muhammad et al. [9] proposed two countermeasures against Bitcoin DDoS mempool attacks: (fee-based mempool and Age-based mempool). The core concept is to use Lyapunov optimization to limit mempool size by detecting fraudulent transactions using the mining fee, the relay fee, and the transaction age. This approach can withstand DDoS attacks while increasing the availability of blockchain nodes. As a result, it meets the *Av* criteria.

B. Consensus Attacks Solutions

Heilman's scheme used random beacons to generate unforgeable timestamps [10]. This immutable timestamp is intended to penalize miners who hold mined blocks and reduce the profits of selfish mining cartels. The proposed approach raises the threshold for the selfish cartel to launch a selfish mining attack from 25% to 32% of the mining power. With its threshold and unforgeable timestamps, the FP system can guarantee that trustworthy nodes mine transactions without fear of forks resulting from selfish mining. This scheme secures the *In*, at the level of consensus, and reduces selfish mining and timestamp

dependency attacks on block timestamps. In order to meet the security requirements, *In* and *Nr* are satisfied.

Olat and Potop-Butucaru [11] presented a zero-block algorithm to counter selfish mining attacks without timestamps. A fixed time interval is generated by averaging the nonce solution and the information dissemination times. Honest nodes reject blocks after a set time and construct a dummy block to alert others. After the virtual blocks are released, this time interval filters and rejects block broadcasts. This technique resists selfish mining pools better than Hellman. If a selfish miner does not tamper with a transaction within a specific time, a double-spend attack is unlikely. Limiting the time needed to mine blocks legally satisfies the *Int* and *Nr* criteria of consensus layer transactions.

Jaewon and Lim [12] proposed a method for randomly selecting mining groups, instead of all miners cooperating to locate the next block. The system divides hashing power into groups and randomly assigns the next block to these groups. This lowers the adversary's hashing power, rendering them incapable of attacking. This approach minimizes the probability of selfish mining and 51% attack. Selecting random nodes for mining purposes could prevent 51% attacks and selfish mining. Additionally, it satisfies *Nr* and *Int* of the specified security requirements for the consensus layer.

Yang et al. [13] proposed a new mechanism to mitigate a 51% attack on PoW consensus by increasing the minimal cost of the attack. This approach computes Historical Weighted Difficulty (HWD) by using the miner's block frequency historical weight. The HWD decides whether or not a branch switch is needed to choose miners. In this scheme, miners always sign the new block with their private key to ensure non-repudiation. This work is ineffective in terms of protecting the system; it just delays the attack and satisfied *Nr* criteria.

C. Smart Contract Attack Solutions

After deployment, significant effort is required to update the source code of the smart contract. There is no way to change a smart contract once it has been deployed on the blockchain. This subsection will evaluate research papers that mostly use fuzzy approaches, symbolic execution methods, and formal verification to verify the correctness of smart contracts and find vulnerabilities.

1) Fuzzing method

Liu et al. [14] proposed a fuzzy-based dynamic analyzer to detect the reentrancy flaw in smart contracts. This analyzer converts smart contracts to equivalent C++ code and dynamically identifies reentrancy vulnerabilities using a fuzzy approach. This tool can only detect reentrancy errors and is incapable of evaluating the most common smart contract errors.

Jiang et al. [15] scheme used a fuzzy technique to produce input for vulnerability analysis related to smart contract application binary interface (ABI) files. Unfortunately, it has many false positives for specific vulnerabilities (such as timestamp dependency). Xiupei et al. [16] advanced this framework using a test oracle. This

scheme uses the White-Box approach, which is more accurate. These analysts found Ethereum vulnerabilities such as exception disorder, reentrancy, timestamp dependency, unsafe delegate call, and freezing ether. This framework alerts pre-deployment smart contract vulnerabilities.

Ding et al. [17] presented a fuzzing-based method for detecting Hyperledger Fabric smart contract vulnerabilities. The scheme was created by using Google go-fuzz to generate an initial corpus and Dvyukov go-fuzz to fuzz the Golang smart contracts. Four out of five smart contracts were found to contain three flaws by the proposed method. In addition, a subset of Golang smart contracts contained type conversion errors, logic loopholes, and integer overflows.

2) Symbolic execution method

Tsankov et al. [18] proposed an automated smart contract analyzer. Based on the security properties provided, the framework ensures that the contract's behavior is safe or unsafe. The semantic information of the given smart contract bytecode or source code dependency graph is symbolically encoded to a layered Datalog in the scheme. These patterns efficiently detect TOD, reentrancy, handling exceptions, and restricted transfers. To some extent, this work ensures the correctness of the contract.

Oyente is a dynamic smart contract analyzer proposed by Luu et al. [19]. This scheme detects TOD, timestamp dependencies, mishandled exceptions, and reentrancy flaws. However, it is prone to produce false-negative results. This false-negative result indicates that this scheme is unable to detect reentrancy bugs at the expected level. As a result, this scheme has limitations in terms of verifying the *Co* of the contract. In another study, [20] extended the Oyente tool to detect previously unknown bugs in smart contracts. The scheme employs static analysis methods and various patterns. Finally, this tool can provide some assurance that the contract is correct.

The scheme proposed by Tikhomirov et al. [21] converts the contract source code into an XML-based parse tree. The converted code then tested against specific XPath patterns to identify vulnerabilities. This scheme detects reentrancy, timestamp dependency, and DOS vulnerabilities. Indeed, the scheme is prone to false-positive rates. This tool ensures the correctness of the contract to some extent.

3) Formal verification method

Bhargavan et al. [22] proposed a formal verification tool to detect three types of Ethereum smart contract vulnerabilities: mishandled exceptions, out-of-gas DoS and reentrancy. The formal verification method is used in this scheme to evaluate contracts and successfully notify the three vulnerabilities mentioned above. Generally, this tool detects three code bugs correctly. As a result, the correctness of the contract can be verified to some extent.

IV. OPEN ISSUES AND FUTURE RESEARCH DIRECTIONS

In this section, several unresolved issues related to blockchain-based smart contract security are highlighted, and a list of future research topics is suggested.

A. Open Research Issues

This section will discuss four open research challenges concerning the security of blockchain-based smart contracts.

First, countermeasures for the security vulnerabilities in each layer are insufficient and uncomprehensive. Instead of conducting a comprehensive study, the researchers addressed each potential attack in the same layer individually. This issue requires the development of a framework to address similar security vulnerabilities in the blockchain.

Second, the miner is able to alter the timestamp in the block, and this issue sometimes compels miners to exploit this vulnerability.

Third, each blockchain transaction is sequentially queued in the miner's memory pool in accordance with the sequential token verification principle. This strategy allows the adversary to weaken the recommended security criteria and launch an attack against the contracts. In this instance, the parallel execution mechanism will be utilized.

Fourth, the smart contract vulnerability analysis tool is in its infant stage, it cannot detect unknown vulnerabilities. Most smart contract analysis tools convert the bytecode or source code of the contract into a compatible language. The converted code may differ from the original smart contract. Multiple analytical tools utilize the contract's source code. However, they are prone to false positives. Consequently, additional research on vulnerability scanning technology is required.

B. Future Research Directions

We further suggest three research directions on blockchain-based smart contracts security according to open issues.

Parallel mining: Executing parallel transactions at runtime is better to implement block creation and verification in transactions. Parallel execution minimizes the chance of attacker miners' chance to destroy tokens queued in the memory pool. Transactions will be executed simultaneously instead of waiting in the memory pool.

Design a new language: The process of finding bugs and validating the code's correctness can be made easier by designing a new programming language. Most of the common smart contract vulnerabilities we have seen in our research originate from the unexpected behavior of programming languages. However, a minor bug in the smart contract could result in a multimillion-dollar loss. As a result, smart contracts must verify that each activity is secure. Functional programming languages, which emphasize using pure functions could be used to construct smart contracts. Smart contracts written in functional programming languages can be analyzed directly without converting them to another language using formal verification methods. As a result, from the beginning, the smart contract code could be rigorously verified with a high level of assurance.

Confidentiality Preservation: The confidentiality of customer data and smart contract workflow is not

adequately considered. Studies have proposed an off-chain infrastructure to execute smart contracts on a secure network. However, off-chain code execution does not guarantee data and code security fully. On-chain code execution with confidentiality must be researched.

V. CONCLUSION

Blockchain has emerged as an ideal platform for smart contract deployment because of its decentralization, immutability, security, and trust. Smart contracts based on blockchain technology are now gaining popularity. However, due to its openness and decentralization, it also faces significant security challenges. We introduced the blockchain-based smart contract platform security model. The requirements for a security countermeasure were then proposed based on the security model. Using the requirements as primary criteria, we thoroughly evaluated the existing literature. Finally, we examined unresolved research issues and offered numerous research directions to stimulate future efforts.

VI. REFERENCES

- [1] N. Szabo, "The idea of smart contracts," *Nick Szabo's papers and concise tutorials*, vol. 6, p. 199, 1997.
- [2] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
- [3] D. Siegel, *Understanding the dao hack for journalists*, Medium, 2016.
- [4] N. Atzei, M. Bartoletti and T. Cimoli, "A survey of attacks on ethereum smart contracts (sok)," in *International conference on principles of security and trust*, 2017.
- [5] M. Alharby, A. Aldweesh and A. Van Moorsel, "Blockchain-based smart contracts: A systematic mapping study of academic research (2018)," in *2018 International Conference on Cloud Computing, Big Data and Blockchain (ICCCBB)*, 2018.
- [6] A. Mense and M. Flatscher, "Security vulnerabilities in ethereum smart contracts," in *Proceedings of the 20th international conference on information integration and web-based applications & services*, 2018.
- [7] S. Rouhani and R. Deters, "Security, performance, and applications of smart contracts: A systematic survey," *IEEE Access*, vol. 7, p. 50759–50779, 2019.
- [8] J. Liu and Z. Liu, "A survey on security verification of blockchain smart contracts," *IEEE Access*, vol. 7, p. 77894–77904, 2019.
- [9] M. Saad, L. Njilla, C. Kamhoua, J. Kim, D. Nyang and A. Mohaisen, "Mempool optimization for defending against DDoS attacks in PoW-based blockchain systems," in *2019 IEEE international conference on blockchain and cryptocurrency (ICBC)*, 2019.
- [10] E. Heilman, "One weird trick to stop selfish miners: Fresh bitcoins, a solution for the honest miner," in *International Conference on Financial Cryptography and Data Security*, 2014.
- [11] S. Solat and M. Potop-Butucaru, "Brief announcement: Zeroblock: Timestamp-free prevention of block-withholding attack in bitcoin," in *International Symposium on Stabilization, Safety, and Security of Distributed Systems*, 2017.
- [12] J. Bae and H. Lim, "Random mining group selection to prevent 51% attacks on bitcoin," in *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2018.
- [13] X. Yang, Y. Chen and X. Chen, "Effective scheme against 51% attack on proof-of-work blockchain with history weighted information," in *2019 IEEE International Conference on Blockchain (Blockchain)*, 2019.
- [14] C. Liu, H. Liu, Z. Cao, Z. Chen, B. Chen and B. Roscoe, "Reguard: finding reentrancy bugs in smart contracts," in *2018 IEEE/ACM 40th International Conference on Software Engineering: Companion (ICSE-Companion)*, 2018.
- [15] B. Jiang, Y. Liu and W. K. Chan, "Contractfuzzer: Fuzzing smart contracts for vulnerability detection," in *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2018.
- [16] X. Mei, I. Ashraf, B. Jiang and W. K. Chan, "A fuzz testing service for assuring smart contracts," in *2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2019.
- [17] M. Ding, P. Li, S. Li and H. Zhang, "Hfcontractfuzzer: Fuzzing hyperledger fabric smart contracts for vulnerability detection," in *Evaluation and Assessment in Software Engineering*, 2021, p. 321–328.
- [18] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Buenzli and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.
- [19] L. Luu, D.-H. Chu, H. Olickel, P. Saxena and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016.
- [20] E. Zhou, S. Hua, B. Pi, J. Sun, Y. Nomura, K. Yamashita and H. Kurihara, "Security assurance for smart contract," in *2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 2018.
- [21] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko and Y. Alexandrov, "Smartcheck: Static analysis of ethereum smart contracts," in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2018.
- [22] K. Bhargavan, A. Delignat-Lavaud, C. Fournet, A. Gollamudi, G. Gonthier, N. Kobeissi, N. Kulatova, A. Rastogi, T. Sibut-Pinote, N. Swamy and others, "Formal verification of smart contracts: Short paper," in *Proceedings of the 2016 ACM workshop on programming languages and analysis for security*, 2016.